

## Java 10 — Local Variable Type Inference (var)

**Java Version:** 10

**Year:** 2018

**Feature:** Local Variable Type Inference (var)

**UniqueClassName:** VarTypeInference

### 1. Definition

**Local variable type inference** was introduced in **Java 10** through the var keyword.

It allows the Java compiler to **infer the type of a local variable from its initializer**, so the programmer does not need to explicitly write the type.

#### Traditional Java

```
String message = "Hello Java";  
int number = 100;
```

#### Java 10

```
var message = "Hello Java";  
var number = 100;
```

The compiler determines:

```
var message = "Hello Java";  
    ↓  
    String
```

```
var number = 100;  
    ↓  
    int
```

**Important:** var does **not** mean dynamically typed. The variable still has a fixed compile-time type.

---

### 2. How var Works

Consider:

```
var name = "Anurag";
```

The compiler analyzes the initializer:

```
"Anurag"  
    ↓
```

String literal



String type



var name → String name

Conceptually, the compiler treats it like:

String name = "Anurag";

Similarly:

var age = 25;

is inferred as:

int age = 25;

And:

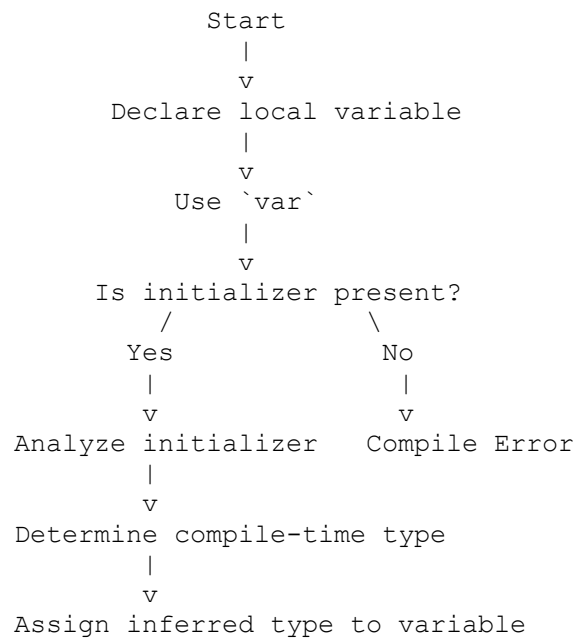
var price = 99.99;

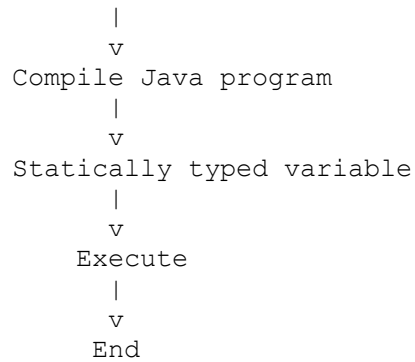
is inferred as:

double price = 99.99;

---

### 3. Flowchart





---

#### 4. Basic Java Code Example

```
public class VarTypeInference {  
  
    public static void main(String[] args) {  
  
        var name = "Java";  
  
        var version = 10;  
  
        var price = 99.99;  
  
        var active = true;  
  
        System.out.println("Name: " + name);  
        System.out.println("Version: " + version);  
        System.out.println("Price: " + price);  
        System.out.println("Active: " + active);  
    }  
}
```

#### Output

```
Name:   Java  
Version: 10  
Price:  99.99  
Active: true
```

The compiler infers:

```
var name      = "Java"; → String  
var version   = 10;     → int  
var price     = 99.99;  → double  
var active    = true;   → boolean
```

---

## 5. var with Objects

var is especially useful when the declared type is long.

### Without var

```
java.util.ArrayList<String> languages =  
    new java.util.ArrayList<String>();
```

### With var

```
var languages =  
    new java.util.ArrayList<String>();
```

The compiler infers:

```
new ArrayList<String>()  
  ↓  
ArrayList<String>  
  ↓  
var languages
```

Example:

```
import java.util.ArrayList;  
  
public class VarTypeInference {  
  
    public static void main(String[] args) {  
  
        var languages = new ArrayList<String>();  
  
        languages.add("Java");  
        languages.add("Python");  
        languages.add("C++");  
  
        System.out.println(languages);  
    }  
}
```

Output:

```
[Java, Python, C++]
```

---

## 6. var with Collections

You can use var with collection objects:

```
var names = java.util.List.of(  
    "Java",  
    "Python",  
    "C++"  
);
```

The compiler infers the actual compile-time type based on the expression.

However, remember that var does **not** automatically mean:

List<String>

It infers the type of the initializer expression according to Java's type rules.

---

## 7. var Is Statically Typed

This is very important.

var does **not** turn Java into a dynamically typed language.

Consider:

```
var number = 100;
```

The compiler determines:

number → int

Therefore, this is invalid:

```
var number = 100;
```

```
number = "Java";
```

because:

```
int  
↓  
cannot become  
↓  
String
```

The compiler reports a type mismatch.

Conceptually:

```
var number = 100
  ↓
  int
  ↓
Fixed compile-time type
  ↓
number = "Java"
  ↓
Compilation Error
```

---

## 8. var Requires an Initializer

This is invalid:

```
var number;
```

Why?

The compiler has no expression from which it can infer the type.

```
var number;
  |
  v
No initializer
  |
  v
No type information
  |
  v
Compilation Error
```

Correct:

```
var number = 100;
```

---

## 9. var Cannot Be Initialized with null

This is invalid:

```
var value = null;
```

null by itself does not provide enough information for the compiler to infer a specific variable type.

```
var value = null
  ↓
  null
  ↓
No specific type
  ↓
Compilation Error
```

---

## 10. var with final

var can be combined with final.

```
final var language = "Java";
```

The compiler infers:

```
final var language = "Java";
  ↓
  final String language
```

The variable cannot be reassigned:

```
final var language = "Java";
```

```
language = "Python"; // Compilation error
```

---

## 11. var in a Loop

var can be used in traditional for loops.

```
public class VarTypeInference {
    public static void main(String[] args) {
        for (var i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

Here:

```
var i = 1
```

↓  
int i = 1

Output:

1  
2  
3  
4  
5

It can also be used with enhanced for loops:

```
var names = java.util.List.of(
    "Java",
    "Python",
    "C++"
);

for (var name : names) {
    System.out.println(name);
}
```

---

## 12. var with Try-With-Resources

var can also be used for local resource variables.

For example:

```
import java.io.BufferedReader;
import java.io.StringReader;
import java.io.IOException;

public class VarTypeInference {

    public static void main(String[] args)
        throws IOException {

        try (var reader =
            new BufferedReader(
                new StringReader("Hello Java"))) {

            System.out.println(reader.readLine());
        }
    }
}
```

The compiler infers the declared type of reader from:

```
new BufferedReader(...)
```

---

### 13. Where var Can Be Used

var is primarily for **local variable declarations**.

#### Allowed

```
var x = 10;
for (var i = 0; i < 10; i++) {
}
for (var item : items) {
}
try (var resource = createResource()) {
}
```

#### Not allowed for fields

```
class Test {
    var value = 10; // Not allowed
}
```

#### Not allowed for method parameters

```
void display(var value) {} // Not allowed
```

#### Not allowed as a method return type

```
var getValue() { // Not allowed
    return 10;
}
```

---

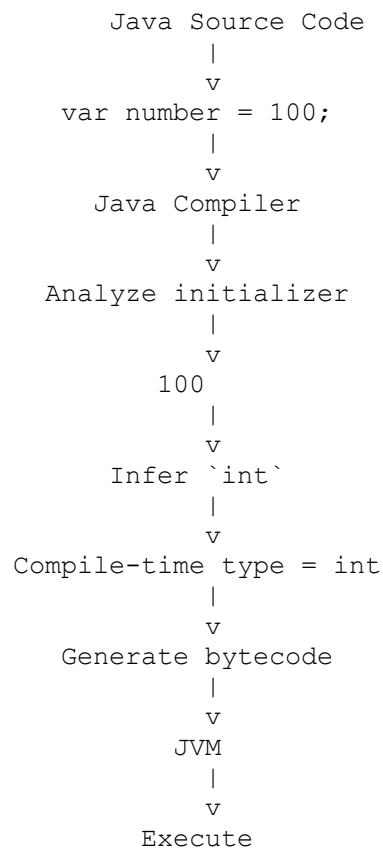
### 14. Traditional Type vs var

| Traditional Java             | Java 10 var        |
|------------------------------|--------------------|
| <b>int</b> number = 10;      | var number = 10;   |
| <b>String</b> name = "Java"; | var name = "Java"; |
| <b>double</b> price = 99.99; | var price = 99.99; |

|                                                                      |                                                        |
|----------------------------------------------------------------------|--------------------------------------------------------|
| <code>ArrayList&lt;String&gt; list = new ArrayList&lt;&gt;();</code> | <code>var list = new ArrayList&lt;String&gt;();</code> |
| Explicit type written                                                | Type inferred by compiler                              |
| Statically typed                                                     | Still statically typed                                 |

---

## 15. Complete Compilation Flow




---

## 16. Important Difference: var vs Dynamic Typing

A common misconception is:

`var` = dynamic typing

This is **incorrect**.

Java remains statically typed:

```
var value = 10;
```

The compiler effectively knows:

value → int

Therefore:

```
value = 20;    // Valid
value = 30;    // Valid
value = "Java"; // Invalid
```

The type is inferred **at compile time**, not determined dynamically at runtime.

---

## 17. Key Concept

The easiest way to remember Java 10 var is:

```
Java 10
|
v
Local variable
|
v
var
|
v
Initializer exists?
|
v
Analyze initializer
|
v
Infer compile-time type
|
v
Statically typed variable
|
v
Bytecode
|
v
JVM
```

### In one sentence

**Java 10's var enables local variable type inference, allowing the compiler to determine a local variable's static type from its initializer while Java remains a statically typed language.**

## Java 10 — AppCDS Improvements

**Java Version:** 10

**Year:** 2018

**Feature:** AppCDS Improvements

**UniqueClassName:** AppCDS

### 1. Definition

**AppCDS** stands for **Application Class-Data Sharing**.

It is an enhancement of the Java Class Data Sharing (CDS) mechanism that allows **application classes to be stored in a shared archive** so they can be reused when the application starts.

Java 10 significantly improved AppCDS by making it possible to archive **application classes from the classpath**, whereas earlier CDS was primarily focused on classes from the JDK.

The main goals are:

- Faster application startup
- Reduced memory usage
- Sharing class metadata between JVM processes
- Reducing repeated class-loading work

### Basic idea

Without AppCDS:

Application Start

|

v

Load application classes

|

v

Parse class files

|

v

Create JVM class metadata

|

v

Run application

With AppCDS:

Application Start

|

v

Load shared archive

|

v  
Reuse archived class metadata  
|  
v  
Load remaining classes  
|  
v  
Run application

---

## 2. CDS vs AppCDS

| Feature              | CDS                               | AppCDS                           |
|----------------------|-----------------------------------|----------------------------------|
| Full name            | Class Data Sharing                | Application Class-Data Sharing   |
| Purpose              | Share class metadata              | Share application class metadata |
| Application classes  | Historically limited              | Supported                        |
| JDK classes          | Yes                               | Yes                              |
| Startup improvement  | Yes                               | Yes                              |
| Memory sharing       | Yes                               | Yes                              |
| Java 10 significance | AppCDS became more broadly usable | Yes                              |

---

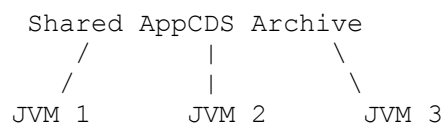
## 3. Why AppCDS Is Useful

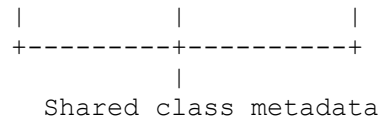
Suppose several Java applications use the same application classes.

Without class sharing:

JVM 1 → Load classes → Create metadata  
JVM 2 → Load classes → Create metadata  
JVM 3 → Load classes → Create metadata

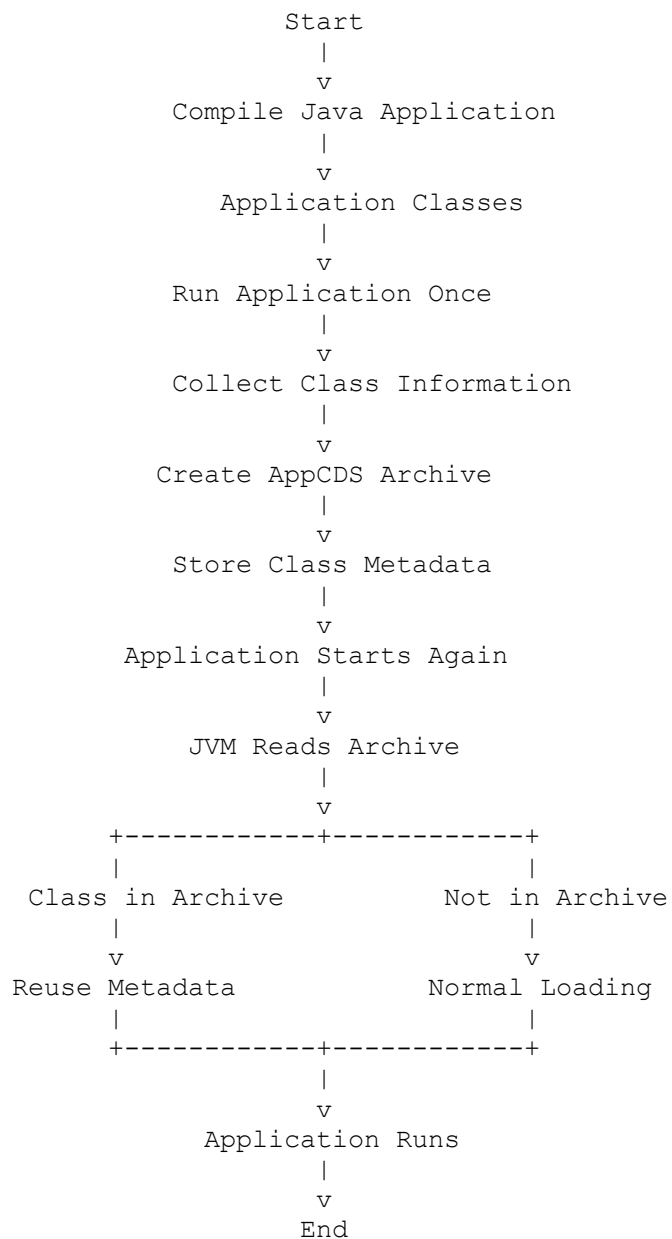
With AppCDS:





The JVM can reuse class metadata from the archive rather than performing all of the same work independently.

#### 4. Flowchart



## 5. Important AppCDS Components

The AppCDS workflow commonly involves:

```
Java Application
|
+---- Application Classes
|
+---- Class List
|
+---- CDS Archive
|
+---- JVM
```

Important JVM options include:

```
-Xshare:off
-Xshare:on
-Xshare:auto

-XX:DumpLoadedClassList=<file>

-XX:SharedClassListFile=<file>

-XX:SharedArchiveFile=<file>

-Xlog:class+load
```

The exact command-line workflow can vary between JDK releases, so commands should be checked against the specific JDK version being used.

---

## 6. Java Code Example

The Java source itself does **not require a special AppCDS API**.

For example:

```
public class AppCDS {

    public static void main(String[] args) {

        System.out.println(
            "Application started successfully."
        );

        System.out.println(
            "AppCDS can share application class metadata."
        );
    }
}
```

```
}  
}
```

Output:

Application started successfully.  
AppCDS can share application class metadata.

The important point is that AppCDS works primarily at the **JVM/class-loading level**, not through Java statements such as:

AppCDS.enable();

There is no such standard Java API.

---

## 7. Compile the Application

For example:

```
javac AppCDS.java
```

This produces:

AppCDS.class

The basic application structure is:

```
AppCDS.java  
  |  
  | javac  
  v  
AppCDS.class
```

---

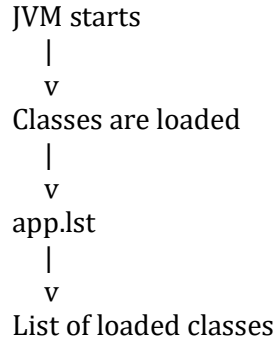
## 8. Generate a Class List

Run the application while asking the JVM to record the classes it loads:

```
java -XX:DumpLoadedClassList=app.lst AppCDS
```

Conceptually:

```
AppCDS.class  
  |  
  v
```



The class list may contain entries corresponding to application and runtime classes that were loaded during that execution.

---

## 9. Create the AppCDS Archive

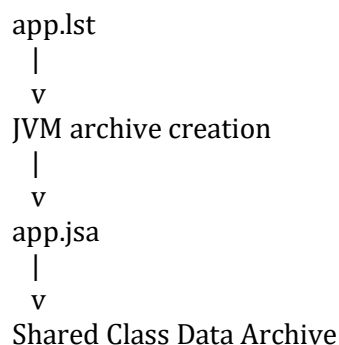
The class list can then be used to create a shared archive.

A typical JDK 10-style command is:

```
java -Xshare:dump \  
-XX:SharedClassListFile=app.lst \  
-XX:SharedArchiveFile=app.jsa \  
-cp . \  
-XX:SharedArchiveConfigFile=app.lst
```

However, **AppCDS command syntax evolved across JDK releases**, so for a practical setup you should use the syntax documented for the exact JDK build you're running.

The conceptual operation is:

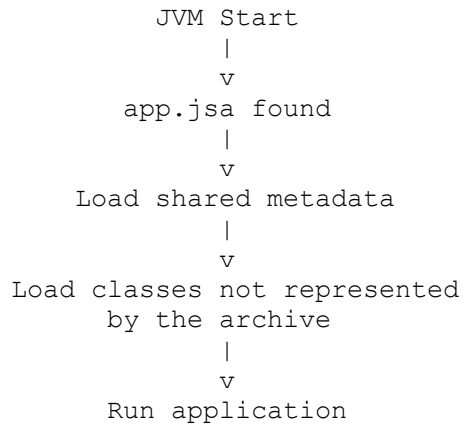


## 10. Running with the Archive

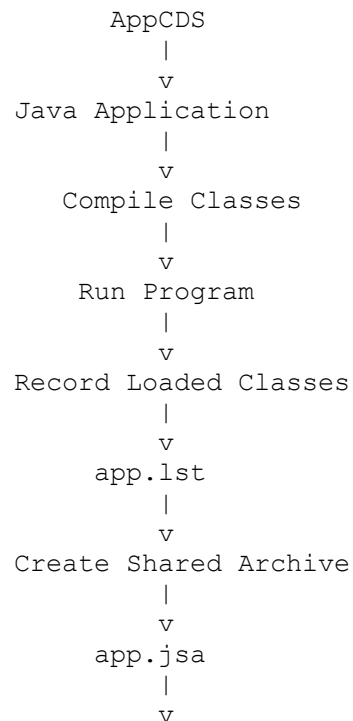
Once an archive has been created, the JVM can be started with the shared archive:

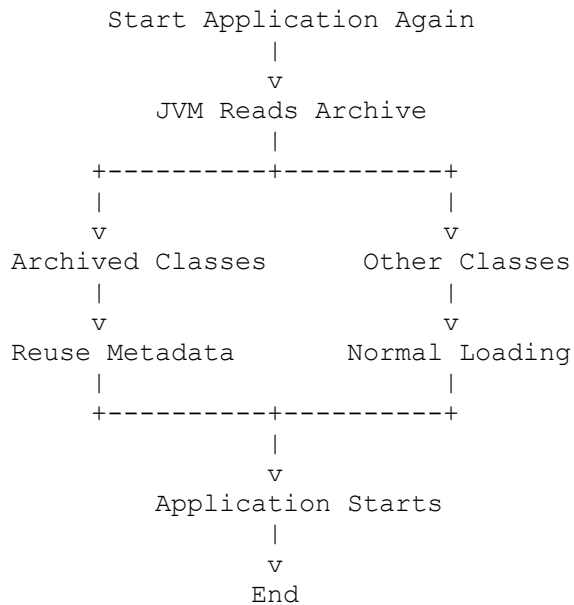
```
java -Xshare:on \  
    -XX:SharedArchiveFile=app.jsa \  
    -cp . \  
    AppCDS
```

Conceptually:



## 11. Complete AppCDS Flow





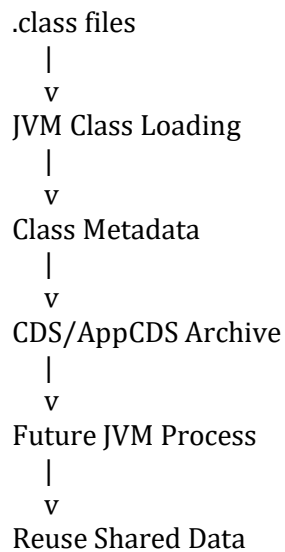
---

## 12. What AppCDS Actually Shares

It is important to understand that AppCDS does **not simply copy .class files into another location**.

The JVM performs work involving class metadata and stores appropriate information in a CDS archive.

Conceptually:



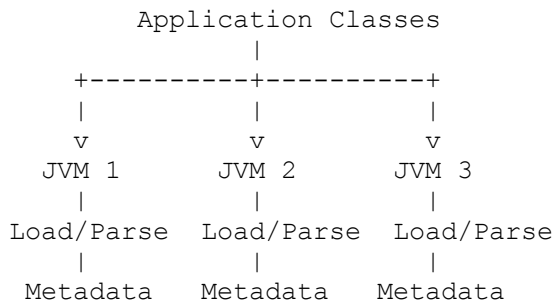
This can reduce repeated class-loading and metadata-generation work.

---

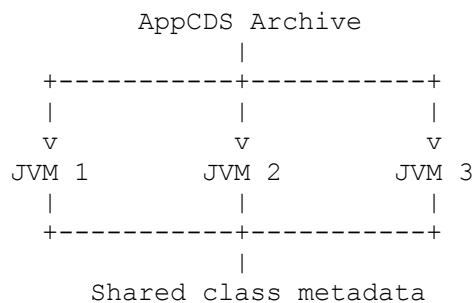
### 13. AppCDS and Multiple JVM Processes

The major benefit becomes clearer when several JVMs run the same application or classes.

#### Without AppCDS



#### With AppCDS



This can improve **startup characteristics and memory efficiency**, particularly when multiple JVM processes use the same classes.

---

### 14. Important Distinction

AppCDS does **not** mean:

"Compile Java code once and execute it directly from the archive."

Instead, think:

Normal Java:

.class  
↓  
Class loading

↓  
JVM metadata

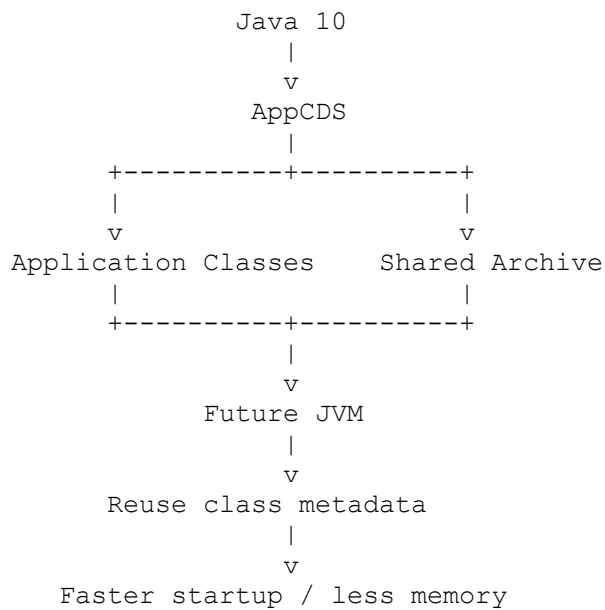
AppCDS:

.class  
↓  
Archived class metadata  
↓  
Future JVM  
↓  
Reuse where applicable

---

## 15. Key Concept

The easiest way to remember Java 10 AppCDS is:



### In one sentence

**AppCDS in Java 10 extended Class Data Sharing to application classes, allowing class metadata to be stored in a shared archive and reused by subsequent JVM processes to reduce class-loading work, improve startup performance, and potentially reduce memory usage.**